

CycleGAN_completed

March 10, 2026

1 PS2.2: Cycle-Consistent Adversarial Networks (CycleGAN)

In this notebook, you will implement the paper [Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks](#) by Zhu et. al.. The problem set focuses on the **apple2orange** dataset because it is visually distinct, and small enough to train reasonably well on a Colab GPU.

Goals:

1. Implement the generator
2. Implement the discriminator
3. Implement the loss functions
4. Translate apples to oranges and back.

1.1 Setup and Imports

```
[1]: import os
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import transforms
import torchvision.utils as vutils
from PIL import Image
import matplotlib.pyplot as plt
import itertools
from tqdm import tqdm
import numpy as np
import random

# Check for GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Set random seeds for reproducibility
torch.manual_seed(42)
np.random.seed(42)
```

Using device: cuda

1.2 Useful Functions for Debugging

```
[2]: def reset_seed(number):  
    """  
    Reset random seed to the specific number  
  
    Inputs:  
    - number: A seed number to use  
    """  
    random.seed(number)  
    torch.manual_seed(number)  
    return  
  
def rel_error(x, y, eps=1e-10):  
    """  
    Compute the relative error between a pair of tensors x and y,  
    which is defined as:  
  
    
$$\text{rel\_error}(x, y) = \frac{\max_i |x_i - y_i|}{\max_i |x_i| + \max_i |y_i| + \text{eps}}$$
  
  
    Inputs:  
    - x, y: Tensors of the same shape  
    - eps: Small positive constant for numeric stability  
  
    Returns:  
    - rel_error: Scalar giving the relative error between x and y  
    """  
    """ returns relative error between x and y """  
    top = (x - y).abs().max().item()  
    bot = (x.abs() + y.abs()).clamp(min=eps).max().item()  
    return top / bot  
  
def count_parameters(model):  
    """Finds the total number of trainable parameters in a model.  
    """  
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

1.3 Download Apple2Orange Dataset

```
[3]: # Download dataset  
if not os.path.exists('datasets/apple2orange'):  
    print("Downloading dataset...")  
    !wget -N https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/apple2orange.  
    ↪ zip -0 apple2orange.zip
```

```

!unzip -q apple2orange.zip -d datasets/
!rm apple2orange.zip
print("Dataset downloaded and unzipped.")
else:
    print("Dataset already exists.")

```

Dataset already exists.

1.4 Dataset Class and Data Loading

```

[4]: class ImageDataset(torch.utils.data.Dataset):
    def __init__(self, root, transforms_=None, mode='train'):
        self.transform = transforms.Compose(transforms_)
        self.files_A = sorted([os.path.join(root, f'{mode}A', x) for x in os.
↪listdir(os.path.join(root, f'{mode}A')) if x.endswith('.jpg')])
        self.files_B = sorted([os.path.join(root, f'{mode}B', x) for x in os.
↪listdir(os.path.join(root, f'{mode}B')) if x.endswith('.jpg')])

    def __getitem__(self, index):
        # CycleGAN uses unpaired data, so we use random indices if lengths
↪differ
        img_A = self.transform(Image.open(self.files_A[index % len(self.
↪files_A)]).convert('RGB'))
        img_B = self.transform(Image.open(self.files_B[index % len(self.
↪files_B)]).convert('RGB')) # Naive pairing for simplicity
        return {'A': img_A, 'B': img_B}

    def __len__(self):
        return max(len(self.files_A), len(self.files_B))

# Hyperparameters
IMAGE_SIZE = 128
BATCH_SIZE = 4

# Transforms
transforms_ = [
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
]

train_dataloader = DataLoader(
    ImageDataset("datasets/apple2orange", transforms_=transforms_,
↪mode='train'),
    batch_size=BATCH_SIZE, shuffle=True, num_workers=2
)

```

```
test_dataloader = DataLoader(
    ImageDataset("datasets/apple2orange", transforms_=transforms_, mode='test'),
    batch_size=BATCH_SIZE, shuffle=False, num_workers=2
)
```

1.5 Visualize the data

```
[5]: reset_seed(42)

vis_batch = next(iter(train_dataloader))
plt.figure(figsize=(24, 30))
plt.subplot(1, 2, 1)
plt.imshow(
    np.transpose(
        vutils.make_grid(
            vis_batch["A"].to(device)[:64], padding=2, normalize=True
        ).cpu(),
        (1, 2, 0),
    )
)
plt.title("Apple")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(
    np.transpose(
        vutils.make_grid(
            vis_batch["B"].to(device)[:64], padding=2, normalize=True
        ).cpu(),
        (1, 2, 0),
    )
)
plt.title("Orange")
plt.axis('off')
plt.show()
```



1.6 Part 1: The Discriminator (PatchGAN)

Implement a discriminator that classifies patches of an image as real or fake. For the discriminator networks we follow the paper to use 70×70 PatchGANs. Let Ck denote a 4×4 Convolution-InstanceNorm-LeakyReLU layer with k filters and stride 2. After the last layer, we apply a convo-

lution to produce a 1-dimensional output. We do not use InstanceNorm for the first $C64$ layer. We use leaky ReLUs with a slope of 0.2. The discriminator architecture is: $C64 - C128 - C256 - C512$.

```
[6]: class Discriminator(nn.Module):
    def __init__(self, in_channels=3):
        super(Discriminator, self).__init__()

        def discriminator_block(in_filters, out_filters, normalize=True,
↪ stride=2):
            """Returns layers of each discriminator block"""
            ↪
            ↪ #####
            ↪ # TODO: Implement the 4x4 Convolution-InstanceNorm-LeakyReLU layer ↪
            ↪ #
            ↪ # with filters and stride 2. ↪
            ↪ #
            ↪ # Hint: Use nn.Conv2d, nn.InstanceNorm2d, nn.LeakyReLU ↪
            ↪ #
            ↪ #####
            ↪ #####

            layers = [
                nn.Conv2d(in_filters, out_filters, 4, stride=stride, padding=1),
                nn.LeakyReLU(0.2, inplace=True),
            ]

            if normalize:
                layers.insert(1, nn.InstanceNorm2d(out_filters))

            ↪
            ↪ #####
            ↪ #
            ↪ # END OF YOUR CODE ↪
            ↪ #
            ↪ #
            ↪ #####
            ↪ #####

            return layers

            ↪
            ↪ #####
            ↪ # TODO: Implement the PatchGAN architecture ↪
            ↪ #
            ↪ # It typically consists of 3-4 discriminator blocks followed by a final ↪
            ↪ convolution. #
            ↪ # Suggested structure: ↪
            ↪ #
```

```

# 1. Block: in_channels -> 64 (normalization=False)
#
# 2. Block: 64 -> 128
#
# 3. Block: 128 -> 256
#
# 4. Block: 256 -> 512
#
# 5. Final Conv2d layer: 512 -> 1 output channel (4x4 kernel,
padding=1)
# Hint: self.model = nn.Sequential(...)
#####

self.model = nn.Sequential(
    *discriminator_block(in_channels, 64, normalize=False),
    *discriminator_block(64, 128),
    *discriminator_block(128, 256),
    *discriminator_block(256, 512),
    nn.ZeroPad2d((1, 0, 1, 0)),
    nn.Conv2d(512, 1, 4, padding=1)
)

#####
#                                     END OF YOUR CODE
#
#####

def forward(self, img):
    return self.model(img)

```

1.7 Discriminator Sanity Check

If your implementation is correct, the number of discriminator's parameters should be 2764737. And the rel_error between the gt_output and test_output should be less than 1e-4.

```

[7]: reset_seed(42)

D_test = Discriminator()
num_params = count_parameters(D_test)
gt_num_params = 2764737
print("GT num_params: ", gt_num_params, ", num_params: ", num_params)

D_test.eval()
with torch.no_grad():

```

```

test_input = torch.ones((1, 3, 32, 32))
test_output = D_test(test_input)

gt_output = torch.tensor([[[-0.0873, -0.4523], [0.1912, -0.0059]]])
print("Error: ", rel_error(gt_output, test_output))

```

GT num_params: 2764737 , num_params: 2764737

Error: 4.1084482743397064e-05

1.8 Part 2: The Generator (ResNet-based)

Implement a ResNet-based generator that transforms images from domain X to Y. For the generator networks we follow the paper's implementation. Let $c7s1-k$ denote a 7×7 Convolution-InstanceNorm-ReLU layer with k filters and stride 1. dk denotes a 3×3 Convolution-InstanceNorm-ReLU layer with k filters and stride 2. Reflection padding was used to reduce artifacts. Rk denotes a residual block that contains two 3×3 convolutional layers with the same number of filters on both layer. uk denotes a 3×3 fractional-strided-ConvolutionInstanceNorm-ReLU layer with k filters and stride $\frac{1}{2}$. The network with 6 residual blocks consists of: $c7s1-64, d128, d256, R256, R256, R256, R256, R256, u128, u64, c7s1-3$

1.8.1 Residual block

we will use the Convolutional Layer with **Reflection Padding** and **Instance Norm**. Architecture for the **ResidualBlock** is described below:

- Reflection Padding (Hint: nn.ReflectionPad2d)
- Convolutional Layer
- InstanceNorm2d (Hint: nn.InstanceNorm2d)
- ReLU
- Reflection Padding (Hint: nn.ReflectionPad2d)
- Convolutional Layer
- InstanceNorm2d (Hint: nn.InstanceNorm2d)

```

[8]: class ResidualBlock(nn.Module):
    def __init__(self, in_features):
        super(ResidualBlock, self).__init__()

        #####
        # TODO: Implement the residual block described above
        #
        # Hint: self.block = nn.Sequential(...)
        #
        #####

        self.block = nn.Sequential(
            nn.ReflectionPad2d(1),
            nn.Conv2d(in_features, in_features, 3),

```

```

        nn.InstanceNorm2d(in_features),
        nn.ReLU(inplace=True),
        nn.ReflectionPad2d(1),
        nn.Conv2d(in_features, in_features, 3),
        nn.InstanceNorm2d(in_features)
    )

    #####
    #                                     END OF YOUR CODE
    #
    #####

    def forward(self, x):
        return x + self.block(x)

```

1.8.2 ResidualBlock Sanity Check

If your implementation is correct, the number of parameters of a ResidualBlock with `in_features=3` should be 168.

```

[9]: test_residual_block = ResidualBlock(in_features=3)
print("GT num parameters: 168, num parameters: ",
      count_parameters(test_residual_block))

```

GT num parameters: 168, num parameters: 168

1.9 Generator

Now let's start building our Generator network. We can divide the whole generator into 5 parts as follows:

1.9.1 Part(a): Initial Convolution

Here we will be implementing the $c7s1 - 64$. Architecture for $c7s1 - 64$ is described below:

- Reflection Padding (Hint: `nn.ReflectionPad2d`)
- Convolutional Layer
- InstanceNorm2d (Hint: `nn.InstanceNorm2d`)
- ReLU

1.9.2 Part(b): Downsampling

Here we will be implementing the $d128$ and $d256$. Architecture for dk is described below:

- Convolutional Layer
- InstanceNorm2d (Hint: `nn.InstanceNorm2d`)
- ReLU

Here we will be adding 9 ResidualBlocks to the generator.

Here we will be implementing the $u128$ and $u64$. Architecture for uk is described below:

- ### ### Part(e): Output Layer

- Reflection Padding (Hint: `nn.ReflectionPad2d`)
- Convolutional Layer
- Tanh

9

```

#
↪#
# Hint:
↪#
# in_features = 64
↪#
# out_features = ...
↪#
# for _ in range(2):
↪#
#     model += ...
↪#
#     in_features = ...
↪#
#     out_features = ...
↪#
↪#####

in_features = 64
for _ in range(2):
    out_features = in_features * 2
    model += [
        nn.Conv2d(in_features, out_features, 3, stride=2, padding=1),
        nn.InstanceNorm2d(out_features),
        nn.ReLU(inplace=True)
    ]
    in_features = out_features

↪#####
#                                     END OF YOUR CODE
↪#
↪#####

↪#####
# TODO: Implement Part(c) Adding the ResidualBlocks.
↪#
↪#####

for _ in range(num_residual_blocks):
    model += [ResidualBlock(in_features)]

```

```

    #####
    #                                     END OF YOUR CODE
    #
    #####

    #####
    # TODO: Implement Part(d) Upsampling.
    #
    #
    # Hint:
    #
    # out_features = ...
    #
    # for _ in range(2):
    #
    #     model += ...
    #
    #     in_features = ...
    #
    #     out_features = ...
    #
    #####

    for _ in range(2):
        out_features = in_features // 2
        model += [
            nn.ConvTranspose2d(in_features, out_features, 3, stride=2,
padding=1, output_padding=1),
            nn.InstanceNorm2d(out_features),
            nn.ReLU(inplace=True)
        ]
        in_features = out_features

    #####
    #                                     END OF YOUR CODE
    #
    #####

```

```

#####
# TODO: Implement Part(e) Output Layer.
#
#####

model += [
    nn.ReflectionPad2d(3),
    nn.Conv2d(in_features, 3, 7, stride=1),
    nn.Tanh()
]

#####
#                                END OF YOUR CODE
#
#####

self.model = nn.Sequential(*model)

def forward(self, x):
    return self.model(x)

```

1.10 Generator Sanity Check

If your implementation is correct, the number of generator's parameters should be 7837699.

```

[11]: reset_seed(42)

test_generator = Generator()
print("GT num parameters: 7837699, num parameters: ",
      count_parameters(test_generator))

```

GT num parameters: 7837699, num parameters: 7837699

1.11 Part 3: Loss Functions

Implement the specific loss logic for CycleGAN.

1.11.1 Generator Loss Functions

The Generator Loss is composed of 3 parts. They are described below:

1. **Adversarial Loss:** It is the standard generator loss of the Least Squares GAN. We use the functional forms of the losses to implement this part. Please refer to the written problem PS.2.4.c for more details.

$$L_{GAN} = ((D_A(G_{B2A}(Image_B)) - 1)^2 + (D_B(G_{A2B}(Image_A)) - 1)^2)$$

2. **Cycle Consistency Loss:** This loss computes the similarity of the original image and the image generated by a composition of the 2 generators. This allows cyclegan to deal with unpaired images. We reconstruct the original image and try to minimize the L_1 norm between the original images and this reconstructed image.

$$L_{cycle_consistency} = (\|G_{B2A}(G_{A2B}(Image_A)) - Image_A\|_1 + \|G_{A2B}(G_{B2A}(Image_B)) - Image_B\|_1)$$

3. **Identity Loss:** It is a regularizing term used in CycleGAN to encourage the generator to preserve the color, tint, and composition of the input image when it already belongs to the target domain.

$$L_{identity} = (\|G_{B2A}(Image_A) - Image_A\|_1 + \|G_{A2B}(Image_B) - Image_B\|_1)$$

1.11.2 Adversarial Loss Implementation

```
[12]: def generator_loss(logits_fake):  
    """  
    Computes the generator loss described above.  
  
    Inputs:  
    - logits_fake: PyTorch Tensor of shape (N,) giving scores for the fake data.  
  
    Returns:  
    - loss: PyTorch Tensor containing the (scalar) loss for the generator.  
    """  
    loss = None  
  
    # TODO: Implement generator_loss.  
    #  
  
    loss = torch.mean((logits_fake - torch.ones_like(logits_fake))*2)  
  
    # END OF YOUR CODE  
    #  
  
    return loss
```

1.11.3 Adversarial Loss Sanity Check

Test your implementation by running the cell below.. You should see errors $< 1e-5$.

```
[13]: logits_fake = torch.tensor(
      [-1.80865868,  0.09030055, -0.4428902 , -0.07879368, -0.37655044,
```

Error: 2.9183900672010386e-06

1.11.4 Cycle Consistency Loss Implementation

```
[14]: def cycle_consistency_loss(real_image, reconstructed_image):
    """
    Computes the cycle consistency loss (L1 norm) between a real image
    and its reconstructed version after a full cycle.

    Inputs:
    - real_image (torch.Tensor): The original image from Domain A or B.
    - reconstructed_image (torch.Tensor): The image after G_AB then G_BA.

    Returns:
    - loss: torch.Tensor L1 loss.
    """
    loss = None

    # TODO: Implement cycle_consistency_loss.

    #

    loss = torch.mean(torch.abs(reconstructed_image - real_image))

    #
    END OF YOUR CODE
    #

    return loss
```

1.11.5 Cycle Consistency Loss Sanity Check

Test your implementation by running the cell below.. You should see errors $< 1e-4$.

```
[15]: reset_seed(42)
      real_image = torch.randn((1, 3, 64, 64), device=device)
      reconstructed_image = real_image + torch.randn_like(real_image, device=device)
      cycle_consistency_loss_true = torch.tensor(0.8010, device=device)
```

```
print(rel_error(cycle_consistency_loss_true, cycle_consistency_loss(real_image,
↪reconstructed_image)))
```

8.311217830025104e-05

1.11.6 Identity Loss Implementation

```
[16]: def identity_loss(real_target_domain, identity_generated_image):
    """
    Computes the identity loss (L1 norm) for CycleGAN.

    Inputs:
    - real_target_domain (torch.Tensor): A real image from the target domain (e.
    ↪g., Domain B).
    - identity_generated_image (torch.Tensor): The output of the generator G_AB
    ↪when
    ↪Domain B as input.
    ↪given the real image from
    ↪Domain B as input.

    Returns:
    - loss: torch.Tensor L1 identity loss.
    """
    loss = None

    ↪
    ↪#####
    ↪# TODO: Implement identity_loss.
    ↪#
    ↪
    ↪#####

    loss = torch.mean(torch.abs(identity_generated_image - real_target_domain))

    ↪
    ↪#####
    ↪#
    ↪#
    ↪#
    ↪#####

    return loss
```

1.11.7 Identity Loss Sanity Check

Test your implementation by running the cell below.. You should see errors < 1e-4.

```
[17]: reset_seed(42)
real_target_domain = torch.randn((1, 3, 64, 64), device=device)
identity_generated_image = torch.randn((1, 3, 64, 64), device=device)
```

2.1439679777787753e-05

$$L_{GAN} = \frac{1}{2} \times \left(((D_A(Image_A) - 1)^2 + (D_A(G_{B2A}(Image_B))^2) + ((D_B(Image_B) - 1)^2 + (D_B(G_{A2B}(Image_A))^2) \right)$$


```

    0.32084742, -0.28590837,  1.01376281,  0.99241439,  0.39394346],  

    ↪device=device)  

logits_real = torch.tensor(  

    [ 0.93487311, -1.01698916, -0.57304769, -0.88162704, -1.40129389,  

    -1.45395693, -1.54239755, -0.57273325,  0.98584429,  0.13312152],  

    ↪device=device)  

discriminator_loss_true = torch.tensor(1.8770, device=device)  

print(rel_error(discriminator_loss_true, discriminator_loss(logits_real,  

    ↪logits_fake)))

```

7.843490918698248e-06

1.11.10 Total Loss Function

The total loss function is defined as:

$$L = L_{GAN} + \lambda \times L_{cycle_consistency} + 0.5\lambda \times L_{identity}$$

The hyperparameter λ is set to 10.

1.12 Training Loop

We provide you the main training loop... you won't need to change this function, but we encourage you to read through and understand it.

```

[20]: reset_seed(42)

# Hyperparameters
lr = 0.0001
n_epochs = 200
Lambda = 10

# Initialize models
G_AB = Generator().to(device) # Apple -> Orange
G_BA = Generator().to(device) # Orange -> Apple
D_A = Discriminator().to(device) # Classifies real/fake Apple
D_B = Discriminator().to(device) # Classifies real/fake Orange

# Optimizers
optimizer_G = torch.optim.Adam(itertools.chain(G_AB.parameters(), G_BA.  

    ↪parameters()), lr=lr)
optimizer_D_A = torch.optim.Adam(D_A.parameters(), lr=lr * 0.1)
optimizer_D_B = torch.optim.Adam(D_B.parameters(), lr=lr * 0.1)

# LR Schedulers
scheduler_G = torch.optim.lr_scheduler.LinearLR(optimizer_G, start_factor=1.0,  

    ↪end_factor=0.0, total_iters=0.5 * n_epochs)
scheduler_D_A = torch.optim.lr_scheduler.LinearLR(optimizer_D_A, start_factor=1.  

    ↪0, end_factor=0.0, total_iters=0.5 * n_epochs)

```

```

scheduler_D_B = torch.optim.lr_scheduler.LinearLR(optimizer_D_B, start_factor=1.
    ↪0, end_factor=0.0, total_iters=0.5 * n_epochs)

# Lists to log metrics etc.
D_loss_list = []
G_loss_list = []
GAN_loss_list = []
epochnums = []

# Training Loop
for epoch in tqdm(range(1, n_epochs + 1)):
    curr_D_loss_list = []
    curr_G_loss_list = []
    curr_GAN_loss_list = []
    for i, batch in enumerate(train_dataloader):
        real_A = batch['A'].to(device)
        real_B = batch['B'].to(device)

        # -----
        # Train Generators
        # -----
        optimizer_G.zero_grad()

        # GAN Loss (Adversarial)

        fake_B = G_AB(real_A)
        loss_GAN_AB = generator_loss(D_B(fake_B))

        fake_A = G_BA(real_B)
        loss_GAN_BA = generator_loss(D_A(fake_A))

        # Cycle Consistency Loss

        rec_A = G_BA(fake_B)
        rec_B = G_AB(fake_A)
        loss_cycle_A = cycle_consistency_loss(real_A, rec_A)
        loss_cycle_B = cycle_consistency_loss(real_B, rec_B)

        # Identity Loss (for color preservation)

        loss_id_A = identity_loss(real_A, G_BA(real_A))
        loss_id_B = identity_loss(real_B, G_AB(real_B))

        # Total Generator Loss
        loss_G = (loss_GAN_AB + loss_GAN_BA) + Lambda * (loss_cycle_A + ↪
    ↪loss_cycle_B) + 0.5 * Lambda * (loss_id_A + loss_id_B)

```

```

# Train Generator G
loss_G.backward()
optimizer_G.step()

# Train Discriminator A
optimizer_D_A.zero_grad()
loss_D_A = discriminator_loss(D_A(real_A), D_A(fake_A.detach()))
loss_D_A.backward()
optimizer_D_A.step()

# Train Discriminator B

optimizer_D_B.zero_grad()
loss_D_B = discriminator_loss(D_B(real_B), D_B(fake_B.detach()))
loss_D_B.backward()
optimizer_D_B.step()

curr_G_loss_list.append(loss_G.item())
curr_D_loss_list.append(loss_D_A.item() + loss_D_B.item())
curr_GAN_loss_list.append(loss_GAN_AB.item() + loss_GAN_BA.item())

# Update learning rate
if epoch >= 0.5 * n_epochs:
    scheduler_G.step()
    scheduler_D_A.step()
    scheduler_D_B.step()

# Display images/plots/stats
if (epoch - 1) % 10 == 0 or epoch == n_epochs:
    # Print stats
    print(f"[Epoch {epoch}/{n_epochs}] [D loss: {np.mean(curr_D_loss_list):.04f}] [G loss: {np.mean(curr_G_loss_list):.04f}] [GAN loss: {np.mean(curr_GAN_loss_list):.04f}]")
    # Plotting images
    plt.figure(figsize=(18,3))
    images = [real_A, fake_B, real_B, fake_A]
    titles = ['Real Apple', 'Generated Orange', 'Real Orange', 'Generated_
    ↪Apple']
    for i in range(4):
        plt.subplot(1, 5, i+1)
        img = images[i][0].cpu().detach().permute(1, 2, 0) * 0.5 + 0.5
        plt.imshow(img)
        plt.title(titles[i])
        plt.axis('off')
    # Plotting losses
    D_loss_list.append(np.mean(curr_D_loss_list))
    GAN_loss_list.append(np.mean(curr_GAN_loss_list))

```

```

epochnums.append(epoch)
plt.subplot(1, 5, 5)
plt.plot(epochnums, D_loss_list, label="D_loss")
plt.plot(epochnums, GAN_loss_list, label="GAN_loss")
plt.ylim(0.0, 1.2)
plt.title("Loss")
plt.legend(loc='upper left')

plt.show()

```

0%| | 0/200 [00:00<?, ?it/s]

[Epoch 1/200] [D loss: 0.5553] [G loss: 8.8048] [GAN loss: 0.6827]



0%| | 1/200 [00:26<1:26:36, 26.12s/it]

1%| | 2/200 [00:51<1:24:30, 25.61s/it]

2%| | 3/200 [01:16<1:23:45, 25.51s/it]

2%| | 4/200 [01:42<1:23:26, 25.54s/it]

2%| | 5/200 [02:07<1:23:06, 25.57s/it]

3%| | 6/200 [02:33<1:22:48, 25.61s/it]

4%| | 7/200 [02:59<1:22:35, 25.68s/it]

4%| | 8/200 [03:25<1:22:11, 25.68s/it]

4%| | 9/200 [03:50<1:21:47, 25.69s/it]

5%| | 10/200 [04:16<1:21:30, 25.74s/it]

[Epoch 11/200] [D loss: 0.4253] [G loss: 4.5840] [GAN loss: 0.7003]



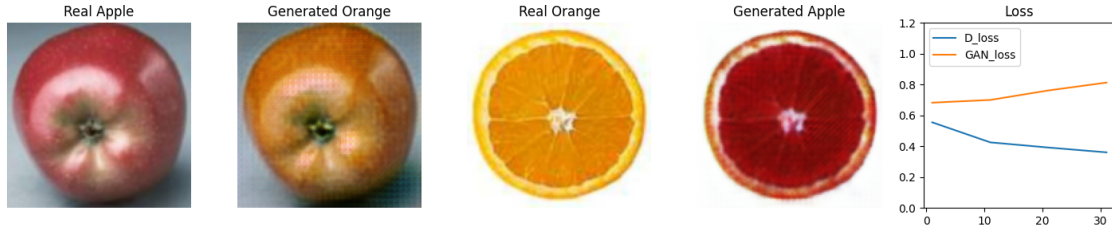
6%| | 11/200 [04:42<1:21:33, 25.89s/it]
6%| | 12/200 [05:08<1:20:59, 25.85s/it]
6%| | 13/200 [05:34<1:20:27, 25.82s/it]
7%| | 14/200 [06:00<1:19:57, 25.79s/it]
8%| | 15/200 [06:25<1:19:27, 25.77s/it]
8%| | 16/200 [06:51<1:18:59, 25.76s/it]
8%| | 17/200 [07:17<1:18:32, 25.75s/it]
9%| | 18/200 [07:43<1:18:05, 25.74s/it]
10%| | 19/200 [08:08<1:17:38, 25.74s/it]
10%| | 20/200 [08:34<1:17:11, 25.73s/it]

[Epoch 21/200] [D loss: 0.3918] [G loss: 4.1059] [GAN loss: 0.7619]



10%| | 21/200 [09:00<1:17:08, 25.86s/it]
11%| | 22/200 [09:26<1:16:34, 25.81s/it]
12%| | 23/200 [09:52<1:16:04, 25.79s/it]
12%| | 24/200 [10:17<1:15:34, 25.76s/it]
12%| | 25/200 [10:43<1:15:05, 25.75s/it]
13%| | 26/200 [11:09<1:14:38, 25.74s/it]
14%| | 27/200 [11:35<1:14:14, 25.75s/it]
14%| | 28/200 [12:00<1:13:47, 25.74s/it]
14%| | 29/200 [12:26<1:13:20, 25.73s/it]
15%| | 30/200 [12:52<1:12:52, 25.72s/it]

[Epoch 31/200] [D loss: 0.3606] [G loss: 3.7096] [GAN loss: 0.8129]



16%| | 31/200 [13:18<1:12:51, 25.87s/it]
 16%| | 32/200 [13:44<1:12:18, 25.82s/it]
 16%| | 33/200 [14:09<1:11:46, 25.79s/it]
 17%| | 34/200 [14:35<1:11:16, 25.76s/it]
 18%| | 35/200 [15:01<1:10:49, 25.76s/it]
 18%| | 36/200 [15:26<1:10:22, 25.75s/it]
 18%| | 37/200 [15:52<1:09:54, 25.74s/it]
 19%| | 38/200 [16:18<1:09:27, 25.73s/it]
 20%| | 39/200 [16:44<1:09:02, 25.73s/it]
 20%| | 40/200 [17:09<1:08:36, 25.73s/it]

[Epoch 41/200] [D loss: 0.3384] [G loss: 3.5174] [GAN loss: 0.8519]



20%| | 41/200 [17:36<1:08:39, 25.91s/it]
 21%| | 42/200 [18:01<1:08:02, 25.84s/it]
 22%| | 43/200 [18:27<1:07:32, 25.81s/it]
 22%| | 44/200 [18:53<1:07:02, 25.78s/it]
 22%| | 45/200 [19:19<1:06:32, 25.76s/it]
 23%| | 46/200 [19:44<1:06:02, 25.73s/it]
 24%| | 47/200 [20:10<1:05:35, 25.72s/it]
 24%| | 48/200 [20:36<1:05:10, 25.73s/it]

24%| | 49/200 [21:01<1:04:44, 25.73s/it]

25%| | 50/200 [21:27<1:04:18, 25.72s/it]

[Epoch 51/200] [D loss: 0.3152] [G loss: 3.4280] [GAN loss: 0.8985]



26%| | 51/200 [21:53<1:04:10, 25.85s/it]

26%| | 52/200 [22:19<1:03:39, 25.81s/it]

26%| | 53/200 [22:45<1:03:11, 25.79s/it]

27%| | 54/200 [23:10<1:02:46, 25.80s/it]

28%| | 55/200 [23:36<1:02:20, 25.79s/it]

28%| | 56/200 [24:02<1:01:51, 25.77s/it]

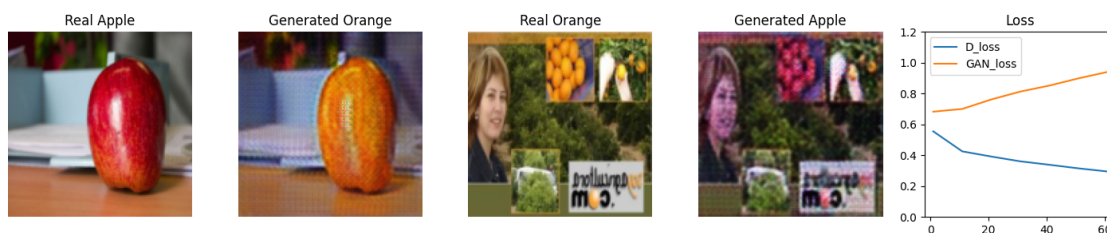
28%| | 57/200 [24:28<1:01:23, 25.76s/it]

29%| | 58/200 [24:53<1:00:55, 25.74s/it]

30%| | 59/200 [25:19<1:00:29, 25.74s/it]

30%| | 60/200 [25:45<1:00:00, 25.72s/it]

[Epoch 61/200] [D loss: 0.2948] [G loss: 3.3489] [GAN loss: 0.9403]



30%| | 61/200 [26:11<59:53, 25.85s/it]

31%| | 62/200 [26:37<59:23, 25.82s/it]

32%| | 63/200 [27:02<58:53, 25.79s/it]

32%| | 64/200 [27:28<58:23, 25.76s/it]

32%| | 65/200 [27:54<57:55, 25.75s/it]

33%| | 66/200 [28:20<57:28, 25.73s/it]

34%| | 67/200 [28:45<57:01, 25.73s/it]

34%| | 68/200 [29:11<56:36, 25.73s/it]

34%| | 69/200 [29:37<56:13, 25.75s/it]

35%| | 70/200 [30:03<55:46, 25.75s/it]

[Epoch 71/200] [D loss: 0.2856] [G loss: 3.2408] [GAN loss: 0.9690]



36%| | 71/200 [30:29<55:38, 25.88s/it]

36%| | 72/200 [30:54<55:07, 25.84s/it]

36%| | 73/200 [31:20<54:36, 25.80s/it]

37%| | 74/200 [31:46<54:09, 25.79s/it]

38%| | 75/200 [32:12<53:42, 25.78s/it]

38%| | 76/200 [32:37<53:15, 25.77s/it]

38%| | 77/200 [33:03<52:49, 25.76s/it]

39%| | 78/200 [33:29<52:21, 25.75s/it]

40%| | 79/200 [33:55<51:54, 25.74s/it]

40%| | 80/200 [34:20<51:28, 25.74s/it]

[Epoch 81/200] [D loss: 0.2779] [G loss: 3.2474] [GAN loss: 1.0013]

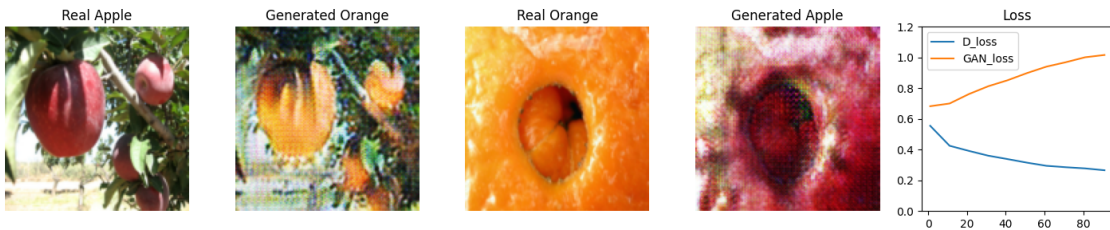


40%| | 81/200 [34:47<51:19, 25.88s/it]

41%| | 82/200 [35:12<50:48, 25.84s/it]

42%| | 83/200 [35:38<50:18, 25.80s/it]
 42%| | 84/200 [36:04<49:50, 25.78s/it]
 42%| | 85/200 [36:30<49:23, 25.77s/it]
 43%| | 86/200 [36:55<48:56, 25.76s/it]
 44%| | 87/200 [37:21<48:28, 25.74s/it]
 44%| | 88/200 [37:47<48:02, 25.74s/it]
 44%| | 89/200 [38:12<47:35, 25.73s/it]
 45%| | 90/200 [38:38<47:10, 25.73s/it]

[Epoch 91/200] [D loss: 0.2658] [G loss: 3.1853] [GAN loss: 1.0163]



46%| | 91/200 [39:04<46:58, 25.86s/it]
 46%| | 92/200 [39:30<46:28, 25.82s/it]
 46%| | 93/200 [39:56<45:59, 25.79s/it]
 47%| | 94/200 [40:21<45:31, 25.77s/it]
 48%| | 95/200 [40:47<45:05, 25.77s/it]
 48%| | 96/200 [41:13<44:39, 25.76s/it]
 48%| | 97/200 [41:39<44:13, 25.76s/it]
 49%| | 98/200 [42:04<43:47, 25.76s/it]
 50%| | 99/200 [42:30<43:20, 25.74s/it]
 50%| | 100/200 [42:56<42:53, 25.74s/it]

[Epoch 101/200] [D loss: 0.2603] [G loss: 3.1545] [GAN loss: 1.0368]



50%| | 101/200 [43:22<42:47, 25.93s/it]
 51%| | 102/200 [43:48<42:14, 25.86s/it]
 52%| | 103/200 [44:14<41:46, 25.84s/it]
 52%| | 104/200 [44:40<41:20, 25.84s/it]
 52%| | 105/200 [45:05<40:53, 25.83s/it]
 53%| | 106/200 [45:31<40:27, 25.82s/it]
 54%| | 107/200 [45:57<40:02, 25.83s/it]
 54%| | 108/200 [46:23<39:36, 25.83s/it]
 55%| | 109/200 [46:49<39:09, 25.82s/it]
 55%| | 110/200 [47:15<38:43, 25.81s/it]

[Epoch 111/200] [D loss: 0.2604] [G loss: 3.0402] [GAN loss: 1.0324]



56%| | 111/200 [47:41<38:29, 25.95s/it]
 56%| | 112/200 [48:07<37:59, 25.91s/it]
 56%| | 113/200 [48:32<37:30, 25.87s/it]
 57%| | 114/200 [48:58<37:04, 25.86s/it]
 57%| | 115/200 [49:24<36:37, 25.85s/it]
 58%| | 116/200 [49:50<36:11, 25.85s/it]
 58%| | 117/200 [50:16<35:44, 25.84s/it]
 59%| | 118/200 [50:42<35:18, 25.84s/it]
 60%| | 119/200 [51:07<34:51, 25.82s/it]
 60%| | 120/200 [51:33<34:24, 25.81s/it]

[Epoch 121/200] [D loss: 0.2669] [G loss: 2.8935] [GAN loss: 1.0030]



60%| | 121/200 [51:59<34:09, 25.94s/it]
61%| | 122/200 [52:25<33:40, 25.91s/it]
62%| | 123/200 [52:51<33:11, 25.87s/it]
62%| | 124/200 [53:17<32:43, 25.84s/it]
62%| | 125/200 [53:43<32:16, 25.83s/it]
63%| | 126/200 [54:08<31:50, 25.82s/it]
64%| | 127/200 [54:34<31:24, 25.81s/it]
64%| | 128/200 [55:00<30:57, 25.80s/it]
64%| | 129/200 [55:26<30:32, 25.81s/it]
65%| | 130/200 [55:52<30:06, 25.81s/it]

[Epoch 131/200] [D loss: 0.2857] [G loss: 2.7525] [GAN loss: 0.9562]



66%| | 131/200 [56:18<29:50, 25.95s/it]
66%| | 132/200 [56:44<29:21, 25.91s/it]
66%| | 133/200 [57:09<28:53, 25.87s/it]
67%| | 134/200 [57:35<28:26, 25.85s/it]
68%| | 135/200 [58:01<27:59, 25.84s/it]
68%| | 136/200 [58:27<27:32, 25.82s/it]
68%| | 137/200 [58:53<27:05, 25.80s/it]
69%| | 138/200 [59:18<26:39, 25.80s/it]

70%| | 139/200 [59:44<26:13, 25.79s/it]

70%| | 140/200 [1:00:10<25:48, 25.81s/it]

[Epoch 141/200] [D loss: 0.3067] [G loss: 2.6143] [GAN loss: 0.8994]



70%| | 141/200 [1:00:36<25:30, 25.94s/it]

71%| | 142/200 [1:01:02<25:02, 25.90s/it]

72%| | 143/200 [1:01:28<24:34, 25.87s/it]

72%| | 144/200 [1:01:54<24:07, 25.85s/it]

72%| | 145/200 [1:02:19<23:41, 25.84s/it]

73%| | 146/200 [1:02:45<23:15, 25.83s/it]

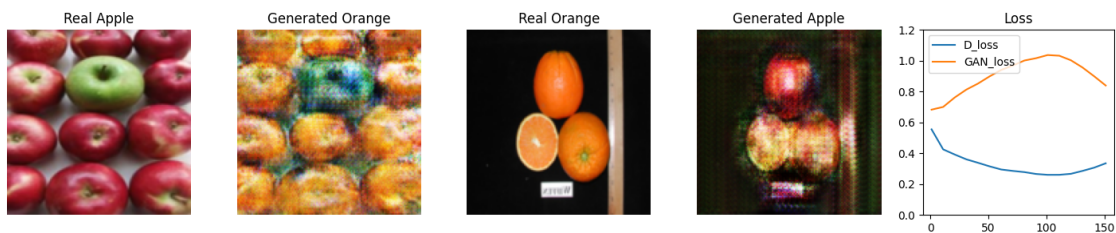
74%| | 147/200 [1:03:11<22:48, 25.82s/it]

74%| | 148/200 [1:03:37<22:22, 25.81s/it]

74%| | 149/200 [1:04:03<21:56, 25.82s/it]

75%| | 150/200 [1:04:29<21:31, 25.83s/it]

[Epoch 151/200] [D loss: 0.3350] [G loss: 2.4909] [GAN loss: 0.8393]



76%| | 151/200 [1:04:55<21:11, 25.95s/it]

76%| | 152/200 [1:05:21<20:43, 25.90s/it]

76%| | 153/200 [1:05:46<20:15, 25.87s/it]

77%| | 154/200 [1:06:12<19:48, 25.83s/it]

78%| | 155/200 [1:06:38<19:20, 25.80s/it]

78%| | 156/200 [1:07:04<18:54, 25.78s/it]

78%| | 157/200 [1:07:29<18:27, 25.77s/it]

79%| | 158/200 [1:07:55<18:01, 25.76s/it]

80%| | 159/200 [1:08:21<17:35, 25.75s/it]

80%| | 160/200 [1:08:47<17:10, 25.75s/it]

[Epoch 161/200] [D loss: 0.3676] [G loss: 2.3713] [GAN loss: 0.7764]



80%| | 161/200 [1:09:13<16:49, 25.88s/it]

81%| | 162/200 [1:09:38<16:21, 25.83s/it]

82%| | 163/200 [1:10:04<15:54, 25.80s/it]

82%| | 164/200 [1:10:30<15:28, 25.79s/it]

82%| | 165/200 [1:10:56<15:02, 25.77s/it]

83%| | 166/200 [1:11:21<14:35, 25.76s/it]

84%| | 167/200 [1:11:47<14:09, 25.75s/it]

84%| | 168/200 [1:12:13<13:44, 25.76s/it]

84%| | 169/200 [1:12:39<13:18, 25.75s/it]

85%| | 170/200 [1:13:04<12:52, 25.74s/it]

[Epoch 171/200] [D loss: 0.4054] [G loss: 2.2584] [GAN loss: 0.7122]



86%| | 171/200 [1:13:31<12:31, 25.92s/it]

86%| | 172/200 [1:13:56<12:04, 25.86s/it]

86%| | 173/200 [1:14:22<11:36, 25.81s/it]
 87%| | 174/200 [1:14:48<11:10, 25.79s/it]
 88%| | 175/200 [1:15:14<10:44, 25.77s/it]
 88%| | 176/200 [1:15:39<10:18, 25.76s/it]
 88%| | 177/200 [1:16:05<09:52, 25.75s/it]
 89%| | 178/200 [1:16:31<09:26, 25.74s/it]
 90%| | 179/200 [1:16:57<09:00, 25.74s/it]
 90%| | 180/200 [1:17:22<08:34, 25.74s/it]

[Epoch 181/200] [D loss: 0.4399] [G loss: 2.1597] [GAN loss: 0.6566]



90%| | 181/200 [1:17:48<08:11, 25.87s/it]
 91%| | 182/200 [1:18:14<07:44, 25.83s/it]
 92%| | 183/200 [1:18:40<07:18, 25.81s/it]
 92%| | 184/200 [1:19:06<06:52, 25.79s/it]
 92%| | 185/200 [1:19:31<06:26, 25.79s/it]
 93%| | 186/200 [1:19:58<06:03, 25.94s/it]
 94%| | 187/200 [1:20:29<05:58, 27.60s/it]
 94%| | 188/200 [1:21:08<06:11, 30.94s/it]
 94%| | 189/200 [1:21:46<06:04, 33.11s/it]
 95%| | 190/200 [1:22:24<05:46, 34.62s/it]

[Epoch 191/200] [D loss: 0.4703] [G loss: 2.0855] [GAN loss: 0.6118]




```

96%|      | 191/200 [1:23:03<05:23, 35.92s/it]
96%|      | 192/200 [1:23:42<04:53, 36.75s/it]
96%|      | 193/200 [1:24:11<04:00, 34.36s/it]
97%|      | 194/200 [1:24:36<03:10, 31.79s/it]
98%|      | 195/200 [1:25:02<02:29, 29.98s/it]
98%|      | 196/200 [1:25:34<02:02, 30.58s/it]
98%|      | 197/200 [1:26:11<01:37, 32.54s/it]
99%|      | 198/200 [1:26:49<01:07, 33.94s/it]
100%|     | 199/200 [1:27:25<00:34, 34.73s/it]

```

[Epoch 200/200] [D loss: 0.4896] [G loss: 2.0422] [GAN loss: 0.5807]



```

100%|     | 200/200 [1:28:02<00:00, 35.45s/it]
100%|     | 200/200 [1:28:02<00:00, 26.41s/it]

```

1.13 Visualization

Show the qualitative results.

```

[21]: def sample_images():
        """Saves a generated sample from the test set"""
        imgs = next(iter(test_dataloader))
        G_AB.eval()
        G_BA.eval()
        real_A = imgs['A'].to(device)
        real_B = imgs['B'].to(device)
        fake_B = G_AB(real_A)
        fake_A = G_BA(real_B)

        # Plotting

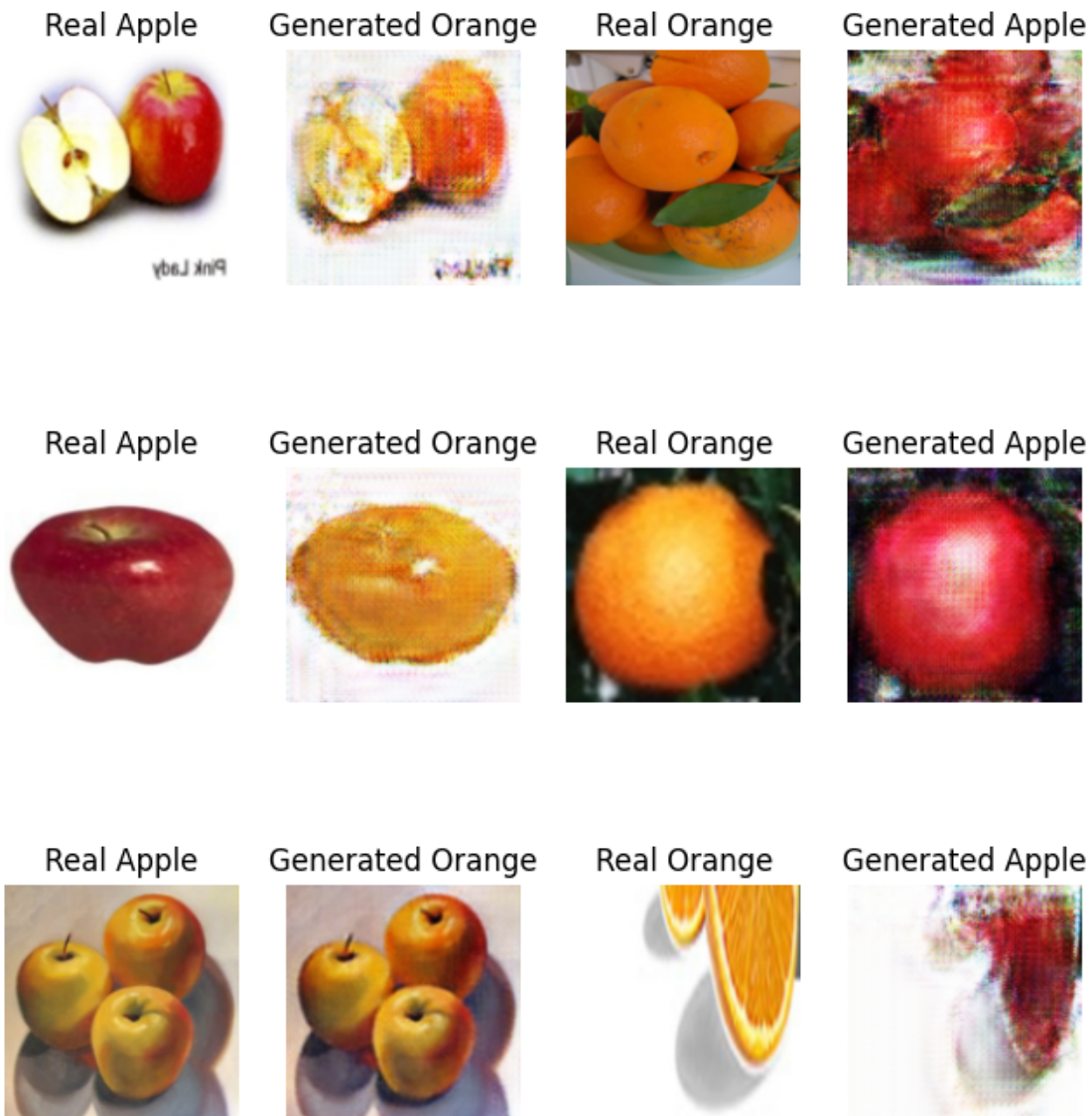
```

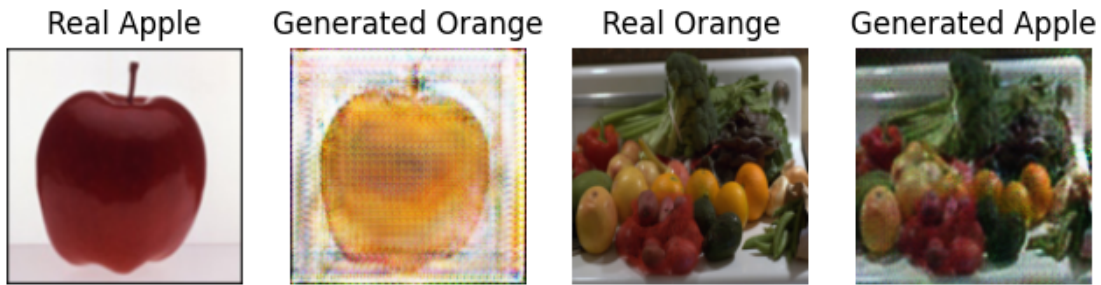
```

images = [real_A, fake_B, real_B, fake_A]
titles = ['Real Apple', 'Generated Orange', 'Real Orange', 'Generated_
↪Apple']
for i in range(BATCH_SIZE):
    plt.figure(figsize=(8, 16))
    for j in range(4):
        plt.subplot(1, 4, j+1)
        img = images[j][i].cpu().detach().permute(1, 2, 0) * 0.5 + 0.5
        plt.imshow(img)
        plt.title(titles[j])
        plt.axis('off')
    plt.show()

sample_images()

```





1.14 Convert Notebook to PDF

```
[22]: notebook_path = './CycleGAN.ipynb' # UPDATE THIS

import os
file_name = notebook_path.split('/')[-1]
get_ipython().system("apt update && apt install texlive-xetex_
↳texlive-fonts-recommended texlive-generic-recommended")
get_ipython().system("pip install py pandoc")
get_ipython().system("apt-get install texlive texlive-xetex texlive-latex-extra_
↳pandoc")
get_ipython().system("jupyter nbconvert --to PDF '{}'.format(notebook_path))
```

Reading package lists... 0%

Reading package lists... 100% Reading package lists... Done

E: Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)

E: Unable to lock directory /var/lib/apt/lists/

W: Problem unlinking the file /var/cache/apt/pkgcache.bin - RemoveCaches (13: Permission denied)

W: Problem unlinking the file /var/cache/apt/srcpkgcache.bin - RemoveCaches (13: Permission denied)

Requirement already satisfied: py pandoc in /home/cash/miniforge3/envs/vallr/lib/python3.10/site-packages (1.16.2)

E: Could not open lock file /var/lib/dpkg/lock-frontent - open (13: Permission denied)

E: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontent), are you root?

Traceback (most recent call last):

```
File "/home/cash/.local/bin/jupyter-nbconvert", line 5, in <module>
    from nbconvert.nbconvertapp import main
```

ModuleNotFoundError: No module named 'nbconvert'